LAPORAN UJIAN AKHIR SEMESTER (UAS)

Diajukan Untuk Memenuhi Tugas Ujian Akhir Semester Mata Kuliah Deep Learning

Nama Mahasiswa	: Arman Surahman
NIM	: 2230511032
Kelas	: 7A Teknik Informatika
Mata Kuliah	: Deep Learning
Dosen Pengampu	: Ibu Asriyanik, M.T.

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS MUHAMMADIYAH SUKABUMI

A. Judul Proyek

Proyek ini berjudul “Klasifikasi Citra Menggunakan Convolutional Neural Network (CNN) pada Dataset CIFAR-10”. Judul ini mencerminkan fokus penelitian, yaitu penerapan metode deep learning berbasis CNN untuk mengklasifikasikan citra digital ke dalam beberapa kelas objek menggunakan dataset CIFAR-10.

B. Latar Belakang

Perkembangan teknologi kecerdasan buatan, khususnya di bidang deep learning, telah mendorong peningkatan signifikan dalam pengolahan dan analisis citra digital. Salah satu permasalahan utama dalam pengolahan citra adalah klasifikasi objek, yaitu proses mengelompokkan citra ke dalam kategori tertentu berdasarkan karakteristik visualnya. Convolutional Neural Network (CNN) merupakan arsitektur deep learning yang dirancang khusus untuk menangani data berbentuk citra dan telah terbukti efektif dalam berbagai tugas klasifikasi visual. Oleh karena itu, proyek ini dilakukan untuk menerapkan CNN dalam melakukan klasifikasi citra menggunakan dataset CIFAR-10 sebagai studi kasus, sekaligus memenuhi tugas Ujian Akhir Semester (UAS).

C. Tujuan Proyek

Tujuan dari proyek ini adalah untuk membangun dan mengevaluasi model Convolutional Neural Network (CNN) yang mampu melakukan klasifikasi citra secara otomatis. Selain itu, proyek ini bertujuan untuk memahami tahapan pra-pemrosesan data, proses pelatihan model, serta evaluasi performa model menggunakan berbagai metrik yang relevan sesuai dengan ketentuan UAS.

D. Dataset yang Digunakan

Dataset yang digunakan dalam proyek ini adalah CIFAR-10, yaitu dataset standar yang berisi 60.000 citra berwarna berukuran 32×32 piksel. Dataset ini terdiri dari 10 kelas objek, seperti pesawat, mobil, burung, kucing, anjing, dan sebagainya. Dataset CIFAR-10 diperoleh melalui library Keras sehingga tidak memerlukan proses unduhan manual. Dataset ini dipilih karena ukurannya relatif kecil, mudah diproses di Google Colab, serta sesuai untuk eksperimen klasifikasi citra menggunakan CNN.

E. Lingkungan Pengembangan dan Tools

Proyek ini dikembangkan menggunakan Google Colab sebagai lingkungan komputasi berbasis cloud. Bahasa pemrograman yang digunakan adalah Python, dengan library utama meliputi TensorFlow/Keras untuk pembuatan model CNN, NumPy untuk pengolahan data numerik, Matplotlib untuk visualisasi, serta Scikit-Learn untuk evaluasi model. Seluruh kode program dijalankan dan disimpan dalam notebook Google Colab.

F. Metodologi

Metodologi yang digunakan dalam proyek ini adalah metode eksperimen dengan pendekatan supervised deep learning. Penelitian dilakukan dengan membangun dan melatih model Convolutional Neural Network (CNN) menggunakan dataset CIFAR-10 yang memiliki label kelas. Tahapan metodologi meliputi pengumpulan dataset, pra-pemrosesan data, perancangan arsitektur CNN, pelatihan model, evaluasi performa model menggunakan berbagai metrik, serta analisis terhadap hasil yang diperoleh.

G. Tahap Pra-Pemrosesan Data dan Persiapan Data

Pada tahap ini, dataset CIFAR-10 dimuat menggunakan fungsi bawaan Keras. Data kemudian dibagi menjadi data training dan data testing. Setiap nilai piksel citra dinormalisasi dengan membagi nilai piksel (0–255) menjadi rentang 0–1 agar proses pelatihan lebih stabil. Label kelas diubah menjadi format one-hot encoding untuk menyesuaikan dengan metode klasifikasi multi-kelas. Tahap pra-pemrosesan ini bertujuan untuk memastikan data siap digunakan oleh model CNN.

```
[4]
✓ 21 d
# Data Loading
from tensorflow.keras.datasets import cifar10
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 11s 0us/step

[5]
✓ 0 d
# Data Cleaning
print(x_train.shape)
print(x_test.shape)
print(y_train.shape)
print(y_test.shape)

(50000, 32, 32, 3)
(10000, 32, 32, 3)
(50000, 1)
(10000, 1)

[6]
✓ 0 d
# Normalisasi Data
x_train = x_train.astype('float32') / 255.0
x_test  = x_test.astype('float32') / 255.0

[7]
✓ 0 d
# Encoding/Tranformasi Data
from tensorflow.keras.utils import to_categorical
y_train = to_categorical(y_train, 10)
y_test  = to_categorical(y_test, 10)
```

H. Perancangan Arsitektur Model CNN

Model CNN dirancang menggunakan beberapa lapisan utama, yaitu convolutional layer, pooling layer, dan fully connected layer. Convolutional layer berfungsi untuk mengekstraksi fitur penting dari citra, sedangkan pooling layer digunakan untuk

mengurangi dimensi data dan mencegah overfitting. Fully connected layer digunakan untuk melakukan klasifikasi akhir berdasarkan fitur yang telah dipelajari. Model dikompilasi menggunakan optimizer Adam dan fungsi loss categorical crossentropy karena permasalahan yang dihadapi adalah klasifikasi multi-kelas.

```
[8]
✓ 0 d
# Arsitektur Model CNN
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)),
    MaxPooling2D(2,2),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax')
])

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:1
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

[9]
✓ 0 d
# Compile Model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

I. Proses Training Model

Model CNN dilatih menggunakan data training dengan jumlah epoch tertentu. Selama proses pelatihan, model mempelajari pola visual dari citra melalui proses forward propagation dan backpropagation. Hasil pelatihan disimpan dalam objek history yang digunakan untuk memantau performa model, termasuk nilai accuracy dan loss pada data training dan data validasi. Proses pelatihan ini bertujuan untuk menghasilkan model yang mampu melakukan generalisasi dengan baik terhadap data yang belum pernah dilihat.

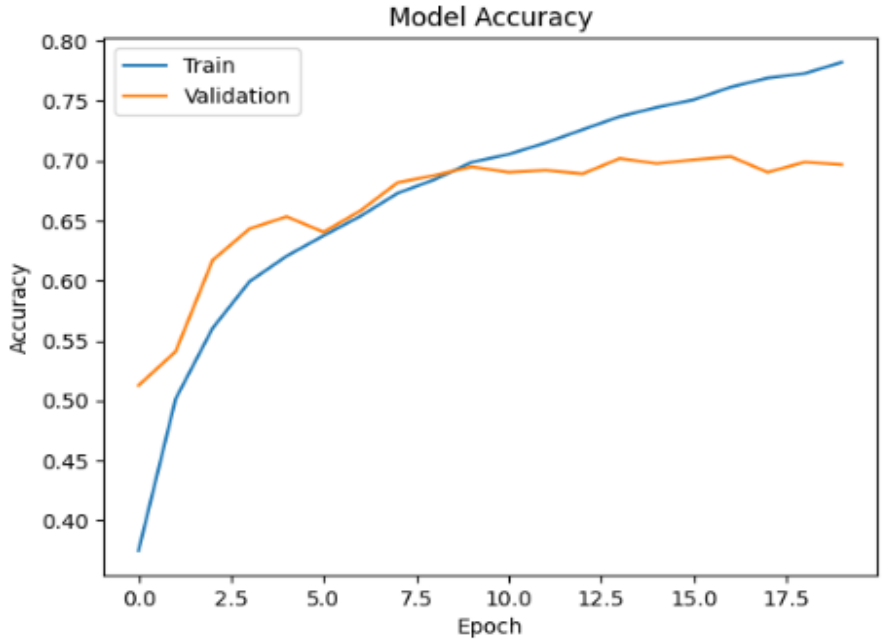
```
[10]
✓ 22m
# Training Model
history = model.fit(
    x_train, y_train,
    epochs=20,
    batch_size=64,
    validation_split=0.2
)
```

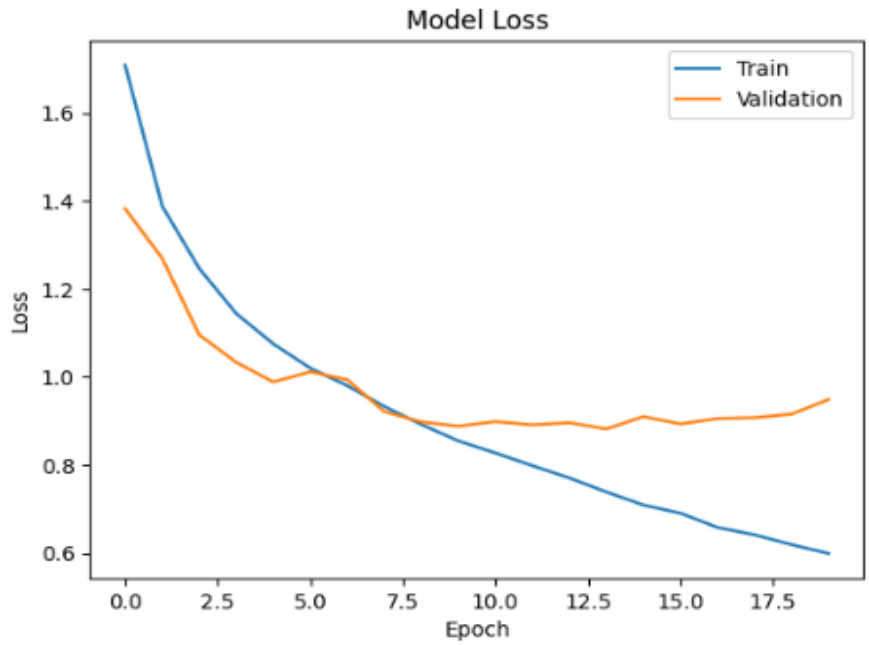
```
Epoch 1/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 54s 84ms/step - accuracy: 0.2927 - loss: 1.9168 - val_accuracy: 0.5127 - val_loss: 1.3821
Epoch 2/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 51s 81ms/step - accuracy: 0.4809 - loss: 1.4335 - val_accuracy: 0.5409 - val_loss: 1.2695
Epoch 3/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 51s 81ms/step - accuracy: 0.5526 - loss: 1.2608 - val_accuracy: 0.6171 - val_loss: 1.0955
Epoch 4/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 50s 79ms/step - accuracy: 0.5986 - loss: 1.1490 - val_accuracy: 0.6434 - val_loss: 1.0333
Epoch 5/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 84s 83ms/step - accuracy: 0.6151 - loss: 1.0857 - val_accuracy: 0.6535 - val_loss: 0.9888
Epoch 6/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 79s 79ms/step - accuracy: 0.6334 - loss: 1.0251 - val_accuracy: 0.6406 - val_loss: 1.0118
Epoch 7/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 81s 77ms/step - accuracy: 0.6577 - loss: 0.9699 - val_accuracy: 0.6585 - val_loss: 0.9937
Epoch 8/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 83s 79ms/step - accuracy: 0.6722 - loss: 0.9309 - val_accuracy: 0.6819 - val_loss: 0.9213
Epoch 9/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 82s 79ms/step - accuracy: 0.6825 - loss: 0.8944 - val_accuracy: 0.6879 - val_loss: 0.8980
Epoch 10/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 81s 78ms/step - accuracy: 0.7029 - loss: 0.8452 - val_accuracy: 0.6951 - val_loss: 0.8882
Epoch 11/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 50s 80ms/step - accuracy: 0.7095 - loss: 0.8151 - val_accuracy: 0.6905 - val_loss: 0.8986
Epoch 12/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 50s 80ms/step - accuracy: 0.7180 - loss: 0.7910 - val_accuracy: 0.6922 - val_loss: 0.8913
Epoch 13/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 52s 83ms/step - accuracy: 0.7303 - loss: 0.7558 - val_accuracy: 0.6892 - val_loss: 0.8961
Epoch 14/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 50s 80ms/step - accuracy: 0.7378 - loss: 0.7338 - val_accuracy: 0.7020 - val_loss: 0.8820
Epoch 15/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 81s 79ms/step - accuracy: 0.7481 - loss: 0.7005 - val_accuracy: 0.6978 - val_loss: 0.9101
Epoch 16/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 84s 83ms/step - accuracy: 0.7538 - loss: 0.6745 - val_accuracy: 0.7009 - val_loss: 0.8933
Epoch 17/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 50s 80ms/step - accuracy: 0.7642 - loss: 0.6472 - val_accuracy: 0.7036 - val_loss: 0.9056
Epoch 18/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 82s 80ms/step - accuracy: 0.7749 - loss: 0.6248 - val_accuracy: 0.6905 - val_loss: 0.9078
Epoch 19/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 50s 80ms/step - accuracy: 0.7776 - loss: 0.6061 - val_accuracy: 0.6990 - val_loss: 0.9157
Epoch 20/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 82s 80ms/step - accuracy: 0.7849 - loss: 0.5854 - val_accuracy: 0.6969 - val_loss: 0.9489
```

J. Visualisasi Learning Curve dan Loss Curve

Untuk menganalisis performa model selama proses pelatihan, dilakukan visualisasi learning curve dan loss curve. Grafik learning curve menampilkan perbandingan akurasi antara data training dan data validasi pada setiap epoch, sedangkan loss curve menunjukkan perubahan nilai loss selama pelatihan. Berdasarkan grafik, terlihat bahwa akurasi training terus meningkat, sementara akurasi validasi cenderung stabil di kisaran tertentu. Hal ini menunjukkan bahwa model mampu belajar dengan baik, meskipun terdapat indikasi overfitting ringan.

```python
# Visualisasi Akurasi
import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(['Train', 'Validation'])
plt.show()

# Visualisasi Loss
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(['Train', 'Validation'])
plt.show()
```





K. Evaluasi Model

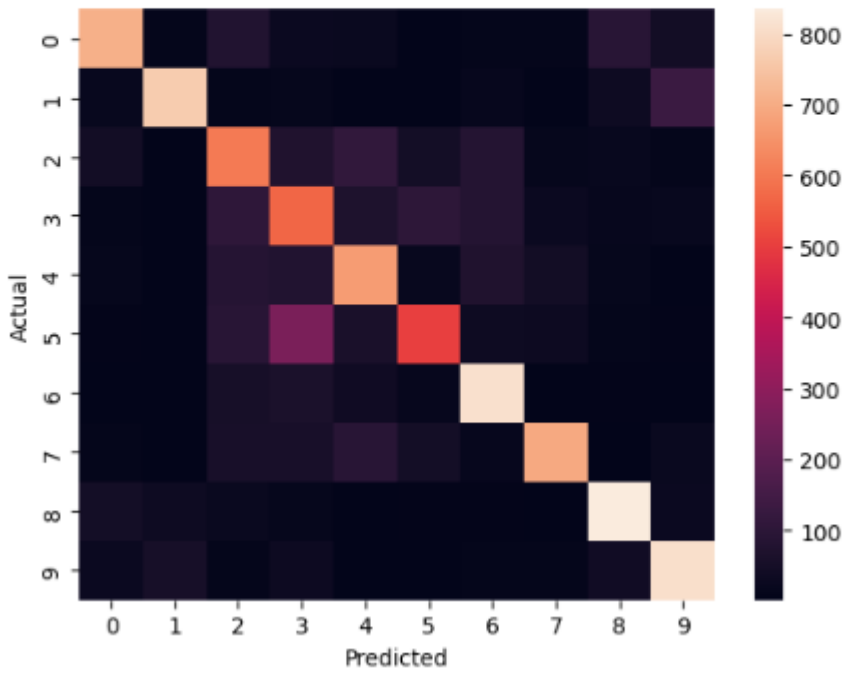
Evaluasi model dilakukan menggunakan data testing yang terpisah dari data training. Metrik evaluasi yang digunakan meliputi accuracy, precision, recall, dan F1-score. Hasil evaluasi menunjukkan bahwa model memperoleh nilai akurasi sekitar 69,6%. Selain itu, classification report menunjukkan nilai rata-rata precision, recall, dan F1-score sebesar 0,70. Confusion matrix digunakan untuk menganalisis kesalahan klasifikasi antar kelas, yang menunjukkan bahwa beberapa kelas memiliki tingkat kesalahan lebih tinggi akibat kemiripan karakteristik visual. Evaluasi ini tidak hanya menilai performa numerik, tetapi juga memberikan pemahaman terhadap kelemahan dan kelebihan model.

```
[11]
4 d
# Evaluasi Model
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test Accuracy:", test_acc)

313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 12ms/step - accuracy: 0.6943 - loss: 0.9492
Test Accuracy: 0.6956999897956848
```

```
[12]
# Prediksi
import numpy as np
y_pred = np.argmax(model.predict(x_test), axis=1)
y_true = np.argmax(y_test, axis=1)

313/313 ━━━━━━━━━━━━━━━━━━━━ 5s 15ms/step
```

```
[13]
# Classification Report
from sklearn.metrics import classification_report
print(classification_report(y_true, y_pred))

...              precision    recall  f1-score   support

           0       0.79      0.71      0.75      1000
           1       0.87      0.77      0.81      1000
           2       0.55      0.60      0.57      1000
           3       0.48      0.57      0.52      1000
           4       0.62      0.67      0.64      1000
           5       0.67      0.50      0.57      1000
           6       0.71      0.81      0.76      1000
           7       0.83      0.69      0.76      1000
           8       0.78      0.83      0.81      1000
           9       0.75      0.81      0.78      1000

    accuracy                           0.70     10000
   macro avg       0.71      0.70      0.70     10000
weighted avg       0.71      0.70      0.70     10000
```

```python
# Confusion Matrix
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_true, y_pred)
sns.heatmap(cm, annot=False)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



L. Analisis Akhir

Berdasarkan hasil evaluasi, model CNN mampu mengklasifikasikan citra CIFAR-10 dengan performa yang cukup baik. Perbedaan antara akurasi training dan validasi menunjukkan adanya overfitting ringan, namun model masih mampu melakukan generalisasi dengan baik. Kesalahan klasifikasi yang terjadi umumnya disebabkan oleh kemiripan visual antar kelas, seperti antara kucing dan anjing atau antara kendaraan tertentu.

M. Kesimpulan

Dari proyek ini dapat disimpulkan bahwa Convolutional Neural Network (CNN) efektif digunakan untuk klasifikasi citra pada dataset CIFAR-10. Model yang dibangun mampu mencapai akurasi sekitar 70% dan menunjukkan performa yang stabil berdasarkan berbagai metrik evaluasi. Proyek ini berhasil memenuhi seluruh kriteria UAS, termasuk tahapan pra-pemrosesan, pelatihan, dan evaluasi model.

N. Saran

Untuk pengembangan selanjutnya, performa model dapat ditingkatkan dengan menambahkan teknik data augmentation, arsitektur CNN yang lebih kompleks, serta penggunaan regularisasi seperti dropout atau batch normalization. Selain itu, eksperimen dengan jumlah epoch dan optimizer yang berbeda juga dapat dilakukan untuk memperoleh hasil yang lebih optimal.

O. Lampiran

Sumber dataset : https://www.cs.toronto.edu/~kriz/cifar.html.

Notebook Coleb : https://colab.research.google.com/drive/1sIXnyv-DG2FwTYYE9aGnXQm9ZyL3MS-E#scrollTo=TedXuxQEQ6vY.

Link Github : ArmanSurahman/cnn-deeplearnig-uas.